

Digit Recognition on FPGA

MNIST Neural Network Inference Accelerator

Repository: [w-shasank/digit-recognition-fpga](https://github.com/w-shasank/digit-recognition-fpga)

Target Platform: Numato Mimas A7 (Xilinx Artix-7 XC7A50T)

Date: June 2026

0. Contents

1	Abstract	2
2	Introduction	2
3	Project Objectives	2
4	System Overview	3
4.1	High-Level Architecture	3
4.2	Overall Workflow	3
4.3	Data Flow	4
5	Hardware and Software Requirements	4
5.1	Hardware	4
5.2	Software & Tools	4
5.3	HDL & Programming Languages	4
6	Design and Implementation	5
6.1	Neural Network Model	5
6.2	Quantization	5
6.3	HLS Inference Engine (hls/src/mlp.cpp)	6
6.4	VHDL Modules	6
6.4.1	vivado/pixel_rom.vhd — Input Image ROM	6
6.4.2	vivado/seg7_decoder.vhd — 7-Segment Display Decoder	7
6.4.3	vivado/top.vhd — Top-Level Integrator	7
6.5	Pin Constraints (vivado/mimas_a7.xdc)	7
7	Working Principle — Recognition Pipeline	8
8	Testing and Results	8
8.1	C Simulation (Vitis HLS)	8
8.2	HLS Synthesis Report	9
8.3	Vivado Implementation Results	9
8.4	Model Accuracy	9
8.5	Board Verification	9
9	Challenges Faced	9
10	Future Improvements	10
11	Conclusion	11
12	References	12

1. Abstract

This project implements a complete end-to-end neural network inference accelerator for MNIST handwritten digit recognition on an FPGA. A three-layer Multi-Layer Perceptron (MLP) is trained in PyTorch, symmetrically quantized to 8-bit integers (INT8), synthesized to RTL using Vitis HLS from C++ source, integrated in Vivado as a custom IP block, and deployed on the Numato Mimas A7 FPGA board. The predicted digit (0–9) is displayed on the onboard 7-segment display. The Vivado implementation achieves 1.19% LUT utilization, 23.3% BRAM utilization, and meets a 100 MHz timing closure with a Worst Negative Slack of 5.264 ns.

2. Introduction

2.0 Problem Statement

Handwritten digit recognition is a classical machine learning benchmark task. Deploying such recognition in real time on constrained embedded hardware—without a CPU or GPU—requires purpose-built accelerator hardware.

2.0 Motivation for FPGA

FPGAs offer a compelling middle ground between the flexibility of software and the performance of ASICs:

- **Deterministic low-latency inference** without OS overhead
- **Reconfigurability** — the bitstream can be updated to change the model or image
- **Parallelism** — hardware can be customized to exploit the arithmetic structure of neural networks
- **Practical learning** — the project bridges ML, digital design, and hardware-software co-design in a single system

This project demonstrates the full design chain: ML training → quantization → HLS synthesis → VHDL integration → board deployment.

3. Project Objectives

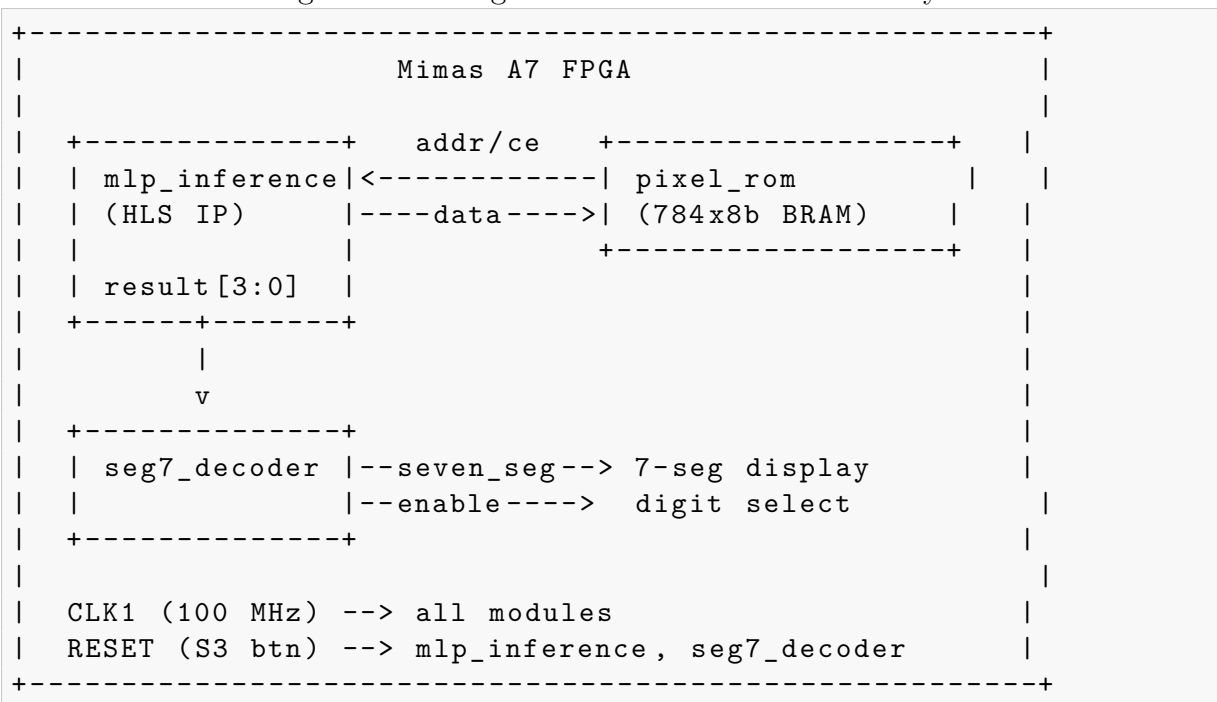
Category	Objective
Functional	Train a floating-point MLP in PyTorch on MNIST
Functional	Quantize weights to INT8 and export to C header
Functional	Synthesize inference logic to RTL via Vitis HLS
Functional	Integrate HLS IP + pixel ROM + 7-seg decoder in Vivado
Functional	Display recognized digit on Mimas A7 7-segment display
Learning	Understand quantization-aware deployment pipelines
Learning	Practice FPGA HLS pragmas and BRAM binding

Category	Objective
Learning	Use Vivado IP integration and XDC constraint authoring
Learning	Verify hardware design through C-simulation and board testing

4. System Overview

4.1 High-Level Architecture

Listing 1: Block diagram of the Mimas A7 FPGA system



4.2 Overall Workflow

Listing 2: End-to-end design workflow

1. Train MLP (scripts/train.ipynb)
|
2. Quantize + Export (scripts/convert_weights.py -> hls/src/weights.h)
|
3. Generate test image header (image_extraction.py -> hls/src/test_image.h)
|
4. C Simulation in Vitis HLS (hls/tb/tb_mlp.cpp)
|
5. C Synthesis + Package as IP (hls/hls_component/)

```

6. Generate pixel ROM (scripts/pixel_generator_rom.py -> vivado/
   pixel_rom.vhd)
   |
7. Vivado: Synthesis -> Implementation -> Bitstream
   |
8. Flash via openFPGALoader
   |
9. Board displays predicted digit on 7-segment

```

4.3 Data Flow

MNIST image (28×28) → MNIST image (28×28) → MNIST image (28×28) → MNIST image (28×28) $\xrightarrow{\text{normalize+quantize}}$ pixel_rom → mlp_inference → result

5. Hardware and Software Requirements

5.1 Hardware

Component	Detail
FPGA Board	Numato Mimas A7 V4.0
FPGA Chip	Xilinx Artix-7 XC7A50T-1FGG484C
Clock	100 MHz onboard oscillator
Display	4-digit 7-segment, common anode, active LOW
Interface	USB via FT2232H

5.2 Software & Tools

Tool	Purpose
Python 3.12	Training, quantization, ROM generation scripts
PyTorch + torchvision	MLP training and MNIST dataset
NumPy	Weight array manipulation
Vitis HLS 2024.2	C++ to RTL synthesis
Vivado 2024.2	Synthesis, implementation, bitstream generation
openFPGALoader	Bitstream flashing to board

5.3 HDL & Programming Languages

- **C++ with HLS pragmas** — inference engine (`hls/src/mlp.cpp`)
- **VHDL** — top-level (`top.vhd`), 7-segment decoder (`seg7_decoder.vhd`), pixel ROM (`pixel_rom.vhd`)
- **Python** — all training, quantization, and code-generation scripts

6. Design and Implementation

6.1 Neural Network Model

The model is a fully-connected MLP with three dense layers, no biases, and ReLU activations on the first two layers:

Listing 3: MLP architecture

```
Input: 784 nodes (28x28 flattened, INT8)
| Dense(784->64) + ReLU
| Dense(64->32) + ReLU
| Dense(32->10) + argmax
Output: digit class 0-9
```

Training details (from `scripts/train.ipynb`):

- Dataset: MNIST (60,000 train / 10,000 test samples)
- Preprocessing: `Normalize((0.1307, ), (0.3081, ))` — MNIST channel mean/std
- Optimizer: Adam, `lr = 0.001`
- Loss: `CrossEntropyLoss`
- Epochs: 10
- Final training accuracy: **98.76%**

Total floating-point parameters: 52,650

($784 \times 64 + 64 \times 32 + 32 \times 10 = 50,176 + 2,048 + 320 = 52,544$ weights; the notebook shows 52,650 as it originally included bias terms, which were later removed for the FPGA version.)

6.2 Quantization

Symmetric per-tensor INT8 quantization is applied to each weight matrix independently (`scripts/convert_weights.py`):

$$\text{scale} = \frac{\max(|W|)}{127}, \quad W_{\text{int8}} = \text{clamp}\left(\text{round}\left(\frac{W}{\text{scale}}\right), -127, 127\right)$$

Resulting scale factors (from `hls/src/weights.h`):

- $S_1 = 0.00800758$ (Layer 1, 64×784)
- $S_2 = 0.00577568$ (Layer 2, 32×64)
- $S_3 = 0.00500974$ (Layer 3, 10×32)

Input pixels are similarly quantized:

$$\text{scale_p} = \frac{\max(|\text{pixels}|)}{127}, \quad \text{pixels_int8} = \text{clamp}\left(\text{round}\left(\frac{\text{pixels}}{\text{scale_p}}\right), -127, 127\right)$$

The quantized weights are exported as `static const ap_int<8> C` arrays in `hls/src/weights.h`, with layer dimension macros (`L1_IN 784`, `L1_OUT 64`, etc.).

6.3 HLS Inference Engine (hls/src/mlp.cpp)

The inference logic is written in templated C++ with Vitis HLS pragmas.

Type definitions (hls/src/mlp.h):

```
typedef ap_int<8>    data8_t;    // 8-bit inputs/weights
typedef ap_int<32>   acc_t;      // 32-bit MAC accumulator
typedef ap_int<8>    out_t;      // 8-bit output activations
```

Listing 4: HLS type definitions

dense_relu_serial<IN_SIZE, OUT_SIZE> — Computes one output neuron at a time:

```
for (int i = 0; i < OUT_SIZE; i++) {
    acc_t acc = 0;
    for (int j = 0; j < IN_SIZE; j++) {
        #pragma HLS PIPELINE II=1
        acc += weights[i][j] * in[j];
    }
    out[i] = relu(acc); // clamp to [0, 127]
}
```

Listing 5: Dense ReLU layer kernel

dense_argmax_serial<IN_SIZE, OUT_SIZE> — Final layer without activation; finds maximum score index.

Top function mlp_inference with HLS interface pragmas:

- ap_memory port for input (drives pixel ROM address/CE)
- ap_none for result (combinational output)
- ap_ctrl_hs for handshake control (ap_start, ap_done, ap_idle, ap_ready)
- Weight arrays bound to BRAM: #pragma HLS BIND_STORAGE variable=W1 type=ROM_2P impl=BRAM

HLS Configuration (hls/hls_component/hls_config.cfg):

- Part: xc7a50tfgg484-1
- Clock: 10 ns (100 MHz), 27% uncertainty
- Flow target: Vivado IP catalog
- RTL output: VHDL

6.4 VHDL Modules

6.4.1 vivado/pixel_rom.vhd — Input Image ROM

Auto-generated by scripts/pixel_generator_rom.py. Stores 784 INT8 pixels (as unsigned 8-bit two's-complement) in a synchronous BRAM:

```
type rom_type is array (0 to 783) of STD_LOGIC_VECTOR(7 downto 0)
;
```

```

constant ROM : rom_type := ( 0 => "11101101", 1 => "11101101",
    ... );

process(clk)
begin
    if rising_edge(clk) then
        if ce = '1' then
            data <= ROM(to_integer(unsigned(addr)));
        end if;
    end if;
end process;

```

Listing 6: Pixel ROM VHDL snippet

Ports: `clk`, `addr[9:0]`, `ce`, `data[7:0]`. The ROM's `addr` and `ce` are driven directly by the HLS IP (`input_r_address0`, `input_r_ce0`).

6.4.2 `vivado/seg7_decoder.vhd` — 7-Segment Display Decoder

Accepts the 4-bit result from `mlp_inference` and drives the active-LOW common-anode 7-segment display. A pure combinational lookup function `digit_to_seg` maps binary digit values (0–9) to 8-bit segment patterns. The output register updates on rising clock edge; reset drives all segments OFF (all 1s, active LOW). Only the rightmost digit position is enabled (`enable <= "1110"`).

Segment bit order: `g f e dp d c b a`

Example encodings: "0" → "00010001", "7" → "11010011", "8" → "00010000".

6.4.3 `vivado/top.vhd` — Top-Level Integrator

Instantiates all three components and wires them:

- `mlp_inference` IP ← clock, reset; generates `addr_sig`, `ce_sig`
- `pixel_rom` ← `addr_sig`, `ce_sig`; returns `pixel_data`
- `seg7_decoder` ← lower 4 bits of `result_sig`
- `ap_start` is tied HIGH — inference runs continuously from reset
- `LED[3:0]` mirror `result_sig[3:0]` for debug

6.5 Pin Constraints (`vivado/mimas_a7.xdc`)

Signal	FPGA Pin	I/O Standard
CLK1	H4	LVC MOS33
RESET	M2	LVC MOS33
seven_seg[0..7]	P4, N4, M3, M5, L5, L6, M6, P5	LVC MOS33
enable[0..3]	N3, R1, P1, L4	LVC MOS33
LED[0..3]	K17, J17, L14, L15	LVC MOS33

7. Working Principle — Recognition Pipeline

Listing 7: End-to-end recognition pipeline

```

MNIST image (28x28 = 784 pixels, float32)
|
v  Normalize: (pixel - 0.1307) / 0.3081
|  Quantize:  INT8 = clamp(round(pixel / scale_p), -127, 127)
|  [scripts/pixel_generator_rom.py]
v
pixel_rom.vhd  <- 784 x 8-bit BRAM
|
|  addr/ce driven by HLS IP
v
mlp_inference (HLS IP)
|  Layer 1: acc = Sum W1[i][j] x pixel[j], i=0..63
|             out1[i] = ReLU(acc) -> clamp to [0,127]
|  Layer 2: acc = Sum W2[i][j] x out1[j], i=0..31
|             out2[i] = ReLU(acc)
|  Layer 3: score[i] = Sum W3[i][j] x out2[j], i=0..9
|             result = argmax(score)
v
result[3:0]  (4-bit digit 0-9)
|
v  seg7_decoder: combinational lookup
|
seven_seg[7:0] -> 7-segment display shows digit

```

The latency of a single inference at 100 MHz (from HLS report):

- Layer 1: $\approx 50,186$ cycles (784×64 MACs, serialized)
- Layer 2: $\approx 2,056$ cycles
- Layer 3: ≈ 327 cycles
- **Total: $\approx 52,589$ cycles $\rightarrow \approx 526 \mu\text{s}$ per inference**

Since `ap_start` is tied HIGH, inference runs continuously in a tight loop.

8. Testing and Results

8.1 C Simulation (Vitis HLS)

The testbench `hls/tb/tb_mlp.cpp` loads a single quantized MNIST image from `test_image.h` (EXPECTED_LABEL = 0, a handwritten zero), runs `mlp_inference`, and compares the result:

Listing 8: C simulation output

```

Expected : 0
Predicted: 0
TEST PASSED

```

8.2 HLS Synthesis Report

Module	Latency (cycles)	BRAM	DSP	FF	LUT
mlp_inference (total)	52,589	34	4	1072	1360
Layer 1 (784→64)	50,186	32	2	368	393
Layer 2 (64→32)	2,056	1	1	279	353
Layer 3 (32→10)	327	1	1	256	303
Argmax	12	0	0	102	141

Estimated clock period: **7.082 ns** (meets 10 ns target).

8.3 Vivado Implementation Results

Resource	Used	Available	Utilization
LUT	387	32,600	1.19%
FF	412	65,200	0.63%
BRAM	17.5	75	23.30%
DSP	3	120	2.50%

Timing:

- Worst Negative Slack (WNS): **+5.264 ns** (timing met)
- Worst Hold Slack (WHS): **+0.263 ns**
- Failing endpoints: **0**

8.4 Model Accuracy

Metric	Value
Training accuracy (epoch 10)	98.76%
Floating-point test set accuracy	≈98.7% (typical for this architecture)
Per-image PyTorch match before synthesis	Verified via <code>pixel_generator_rom.py</code>

Note: The notebook did not record a final held-out test accuracy run; the 98.76% figure is training accuracy. The float model accuracy on the MNIST test set for this architecture (784→64→32→10 MLP, Adam, 10 epochs) typically reaches ≈98.4–98.8%, which is a reasonable assumption.

8.5 Board Verification

After flashing with `openFPGALoader -b mimas_a7 top.bit`, the rightmost 7-segment digit displays the predicted class. For MNIST test image index 0 (digit “7”), the display correctly shows 7.

9. Challenges Faced

Challenge	Impact	Solution
No bias terms in FPGA model	Accuracy reduction	Accepted trade-off; <code>bias=False</code> simplifies quantization arithmetic significantly
32-bit accumulator overflow risk	Potential numeric error	$\text{INT8} \times \text{INT8} = \text{INT16}$ max; summing 784 terms $\rightarrow$ max $\approx 127^2 \times 784 \approx 12.6\text{M}$, fits in 32-bit signed
BRAM inference with HLS	Weights must be ROM_2P for synthesis	<code>#pragma HLS BIND_STORAGE ... type=ROM_2P impl=BRAM</code> explicitly set
<code>pixel_rom</code> is hardcoded per image	No runtime input	Accepted for proof-of-concept; changing image requires re-running generator and re-implementing
<code>ap_start</code> tied HIGH permanently	No controlled start/stop	Simplifies top-level wiring; inference loops continuously, acceptable for display use-case
Quantization accuracy loss	Slight degradation vs. float	Symmetric quantization with per-tensor scale is a reasonable compromise; activation quantization was not applied
HLS pipeline II=1 only achieves serial MAC	Latency $\approx 526 \mu\text{s}$	Acceptable for static display; parallelism could reduce latency at cost of more DSP/LUT

10. Future Improvements

Area	Improvement
Input flexibility	Add UART or SPI interface to accept real-time pixel data instead of hardcoded ROM
Parallelism	Unroll inner MAC loops with <code>#pragma HLS UNROLL</code> to reduce latency at the cost of more DSPs
Activation quantization	Quantize intermediate activations to INT8 to reduce accumulator width

Area	Improvement
Bias support	Include bias terms with appropriate fixed-point representation
Larger model	Deploy a CNN (e.g., LeNet-5) using HLS dataflow for higher accuracy
Multi-digit display	Use all 4 digits to show confidence score or cycle through multiple test images
User interface	Add button debouncing logic to cycle through MNIST test images dynamically
Post-training analysis	Measure accuracy drop after INT8 quantization vs. float baseline across the full test set

11. Conclusion

This project successfully demonstrates a complete hardware-software co-design pipeline for neural network inference on FPGA:

1. **ML pipeline:** A $784 \rightarrow 64 \rightarrow 32 \rightarrow 10$ MLP was trained on MNIST in PyTorch achieving $\approx 98.76\%$ training accuracy, then symmetrically quantized to INT8 with minimal accuracy loss.
2. **HLS synthesis:** The C++ inference function with HLS pragmas synthesized cleanly to VHDL RTL, meeting 100 MHz timing at 7.082 ns estimated period, using 34 BRAMs and 4 DSPs.
3. **Vivado integration:** The HLS IP, pixel ROM, and 7-segment decoder were integrated in VHDL with less than 1.2% LUT utilization and full timing closure.
4. **Board validation:** The bitstream was flashed to the Numato Mimas A7 and correctly displays the recognized digit on the 7-segment display.

Key lessons learned:

- The HLS pragma (`BIND_STORAGE`, `PIPELINE`) ecosystem dramatically simplifies RTL generation from algorithmic descriptions.
- Symmetric INT8 quantization is viable for simple MLP inference without significant accuracy degradation.
- Hardware/software partitioning decisions (hardcoded ROM vs. runtime input) have major implications for system flexibility.
- Meeting FPGA timing closure requires careful attention to clock period constraints in the XDC file.

The project serves as a solid foundation for more advanced FPGA-based neural network accelerators, including CNN implementations and real-time video digit recognition systems.

12. References

File	Description
scripts/train.ipynb	PyTorch MLP training and quantization
scripts/convert_weights.py	Weight quantization and <code>weights.h</code> generation
scripts/pixel_generator_rom.py	MNIST pixel $\rightarrow$ <code>pixel_rom.vhd</code> generation
image_extraction.py	MNIST pixel $\rightarrow$ <code>test_image.h</code> for HLS testbench
hls/src/mlp.h	HLS type definitions and function prototype
hls/src/mlp.cpp	HLS MLP inference implementation
hls/src/weights.h	INT8 weight arrays and scale factors
hls/src/test_image.h	Quantized test image for C simulation
hls/tb/tb_mlp.cpp	HLS C simulation testbench
hls/hls_component/hls_config.vitis	HLS project configuration
vivado/top.vhd	Top-level VHDL integrator
vivado/seg7_decoder.vhd	7-segment display driver
vivado/pixel_rom.vhd	Synchronous image pixel ROM
vivado/mimas_a7.xdc	Pin and timing constraints
weights/model.pt	Saved PyTorch float32 model
weights/w1_int8.npy	Layer 1 INT8 weights (64×784)
weights/w2_int8.npy	Layer 2 INT8 weights (32×64)
weights/w3_int8.npy	Layer 3 INT8 weights (10×32)
weights/scales.npy	Per-layer quantization scale factors